# Lecture 1: Introduction

## Machine Learning 2026

Federico Chiariotti (federico.chiariotti@unipd.it)




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

# Part 1:
# Basic course information




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

# The course

➔ Machine learning (SCP0000228)

If you are from other courses please notify me

This course is worth 6 ECTS (48 hours)

We will have 16 lectures and 8 labs



# Lectures

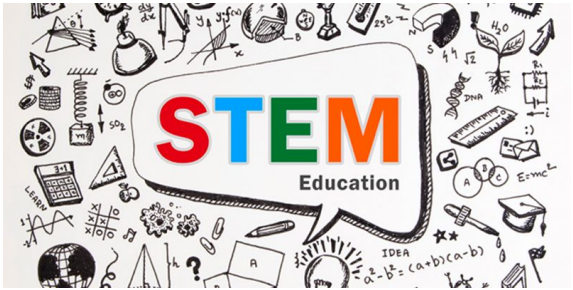
- ➔ Tuesday, 16:30-18:00 – LabP104 (DFA: via Belzoni, 7)
- ➔ Friday, 14:30-16:00 – Room C (DFA: via Paolotti, 9)

Recordings will be available **for 2 weeks only**

# Elearning

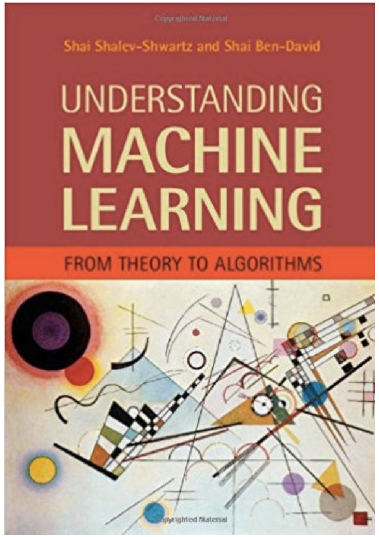
All course materials and announcements will be published through the course Elearning page. The password for manual enrollment is **ML2627PW** – but most of you should be enrolled automatically.

https://stem.elearning.unipd.it/course/view.php?id=17234



# Course material

- Main book: Shalev-Shwartz, Shai, & Ben-David, Shai, *Understanding machine learning: From theory to algorithms*, Cambridge University Press, 2014
- PDF available from the authors at http://www.cs.huji.ac.il/~shais/UnderstandingMachineLearning/copy.html
- Lecture slides: available on Elearning
- Lab solutions and notebooks: available on Elearning



# A new exam format

1. Programming and written exam (2/3)
    a. Open-book programming exam (1h, 8 points) - you will be asked to implement one of the algorithms we study in the labs. You will have access to basic Python documentation, but no Internet access
    b. Closed-book written exam (1h, 24 points) - you will have 3 exercises which you will need to complete without any notes or books
2. Oral exam (1/3) **after passing the first part**

The oral exam will be on the whole course: I may ask for proofs and explanation of the concepts in the course or to review code and check for its correctness.

# Exam rules

1. The first part is a single partial exam, and the written and coding tests must be taken together
2. The results of the first part are valid until the end of the academic year (Sep. 30th) unless you take the oral exam
3. You can take the first part again if you are not satisfied with your grade, but the previous grade will be erased if you take the exam again (even if you withdraw)
4. If you reject a grade or fail the oral exam, you have to take the first part again as well

# Exam rules - first part

1. In the coding test, code that does not run without errors and return a valid result is an automatic fail
2. The coding part may still be a failure if there are fundamental errors, while smaller errors may result in some points being docked
3. In the coding test, you will only be asked to implement the relevant algorithms, the data loading and testing code will be provided (just like in the lab notebooks)
4. In the written test, you will have to solve some exercises in the style of the ones we will see in the course (there is a pdf on the Elearning page with many solved exercises, and I will upload a sample written test)

# Why this format?

- ➔ The objective of the course is to get you to understand the concepts and trade-offs in machine learning applications
- ➔ This includes solving exercises, reasoning about the concepts, and being able to implement the algorithms
- ➔ Past exams clearly showed that, due to AI and its effect on some students' programming skills, the old format did not work and ended up rewarding rote memorization

# External aids during the exam

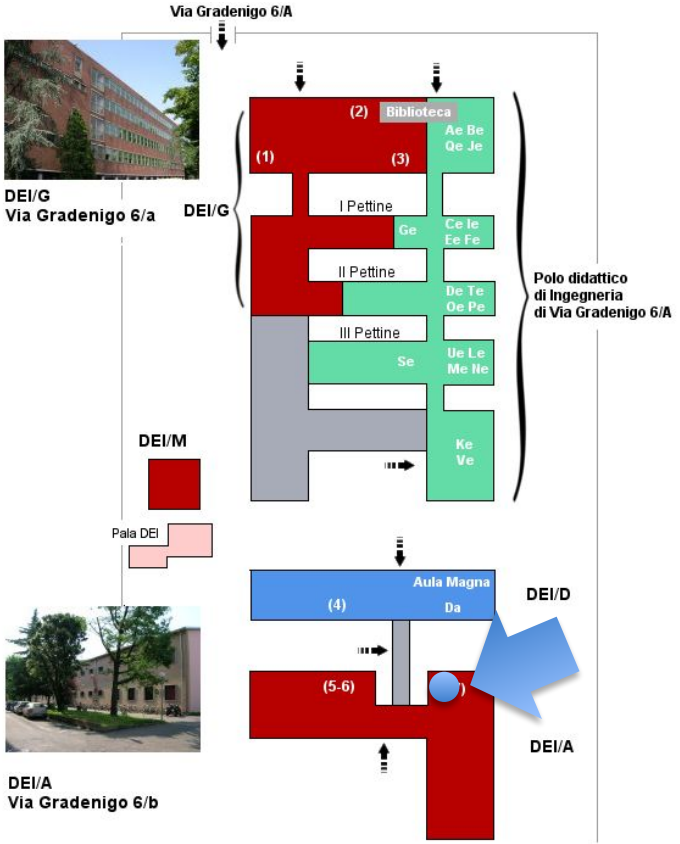
- ➔ Any external tool or aid (notes, books, solved exercises…) is strictly forbidden, and will lead to an automatic fail
- ➔ If you are caught cheating, you will be suspended for the next exam attempt
- ➔ The use of devices such as smartphones and AI tools during the exam will be reported to the university's ethics committee

# The elephant in the room

- ➔ I personally do not use generative AI tools for any aspect of the course
- ➔ The objective of the course is to get **you** to understand and be able to apply machine learning concepts
- ➔ I do not (and cannot) prevent you from using AI to study: I can tell you that it will be harmful if you rely on it too much

# How to reach me

- ➔ Questions during the break or after class
- ➔ Email: federico.chiariotti@unipd.it (I try to reply within a day)
- ➔ If you want to set up a meeting, send me an email. My office is in room 231, DEI/A (via Gradenigo, 6/B)



# Setting up your computer

- ➔ We will use **Jupyter notebooks** as a main tool
- ➔ Recommended: install anaconda3 (https://www.anaconda.com/download)
- ➔ Jupyter notebooks are included by default in Anaconda
- ➔ Useful packages: numpy, pandas, scipy, scikit-learn, matplotlib

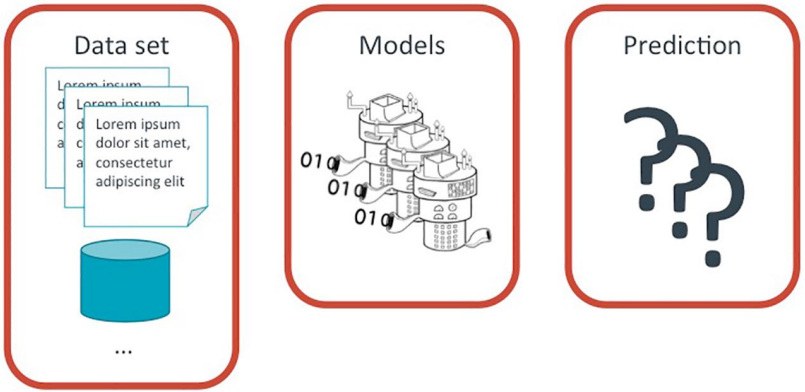


# Running notebooks

- ➔ Command: `jupyter-notebook`
- ➔ The notebook is an interactive environment in your browser
- ➔ If you've never used python, it's very intuitive and simple
- ➔ The "lab 0" experience will help you get familiar with the syntax



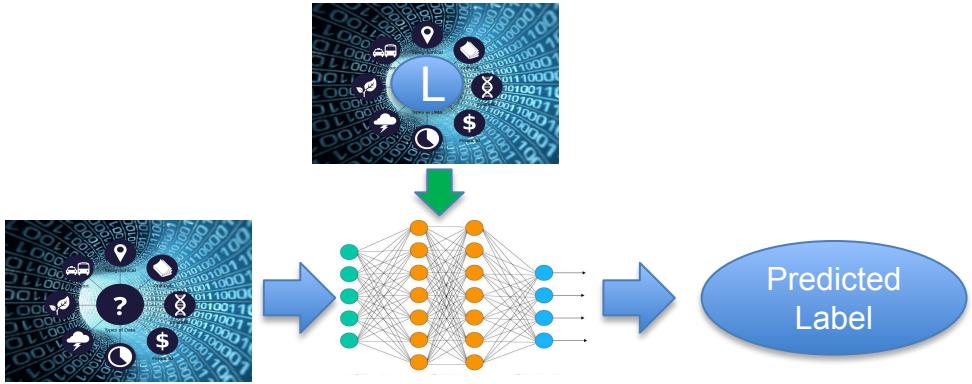
# What is machine learning?




Machine learning (ML) is a set of methods that give computer systems the ability to "*learn*" from (*training*) data to make predictions about novel data samples, *without being explicitly programmed*

# The three families of machine learning problems

1. **Supervised learning**

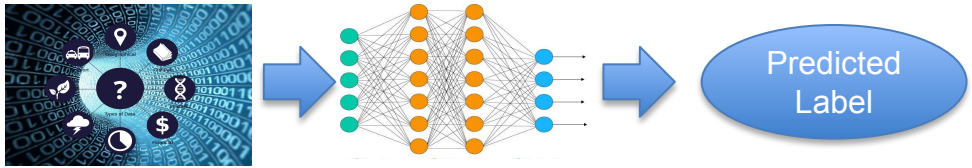



- ➔ Classification
- ➔ Regression
- ➔ Object recognition

# The three families of machine learning problems

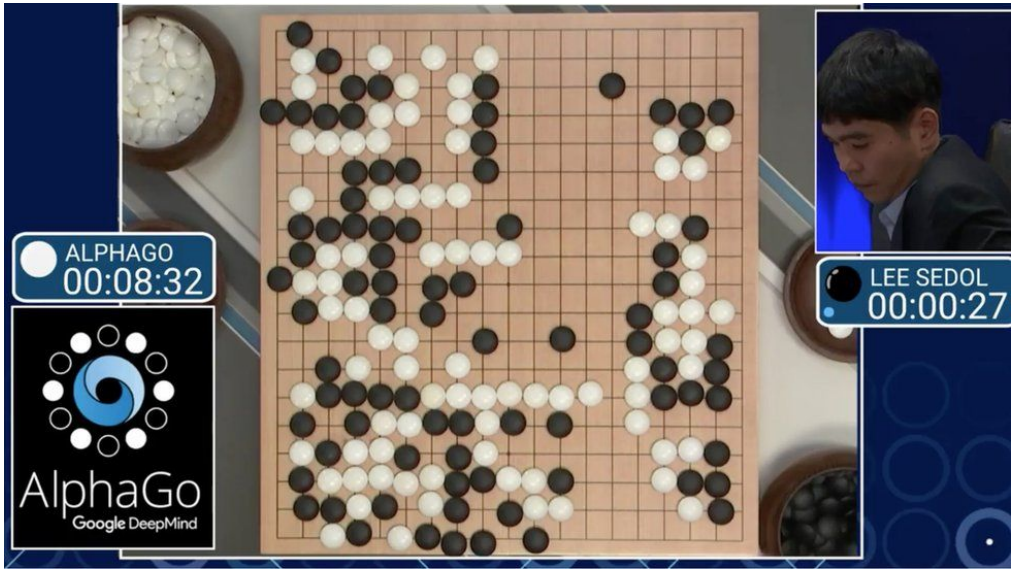
1. Supervised learning
2. **Unsupervised learning**

➔ Clustering
➔ Compression
➔ Generative models



# The three families of machine learning problems

1. Supervised learning
2. Unsupervised learning
3. **Reinforcement learning**




➔ Games
➔ Robotics
➔ Sequential decision-making

# Lecture plan

| Date | # | Topic | Date | # | Topic | Date | # | Topic |
|---|---|---|---|---|---|---|---|---|
| Sep. 29 | 1 | Introduction | Oct. 27 | L2 | *Model selection* | Dec. 1 | L5 | *Random forests* |
| Oct. 2 | 2 | PAC learning | Oct. 30 | 8 | Support Vector Machines | Dec. 4 | 12 | CNNs |
| Oct. 6 | 3 | Uniform convergence | Nov. 3 | L3 | *Linear models* | Dec. 11 | 13 | Clustering models |
| Oct. 9 | 4 | VC dimension | Nov. 6 | 9 | Kernels and random forests | Dec. 15 | L6 | *Backpropagation* |
| Oct. 13 | 5 | Model selection | Nov. 10 | L4 | *SVMs* | Dec. 18 | 14 | Reinforcement learning |
| Oct. 16 | 6 | Linear classification | Nov. 13 | E1 | Exercises | Dec. 22 | L7 | *Clustering* |
| Oct. 20 | L1 | *Python basics* | Nov. 20 | 10 | Boosting and PCA | Jan. 12 | L8 | *RL* |
| Oct. 23 | 7 | Linear regression and optimization | Nov. 27 | 11 | Neural networks | Jan. 15 | E2 | Exercises |

# 1-5: statistical learning

1. Basic formulation: how do we model supervised learning problems?
2. PAC learning: giving performance guarantees on specific problems
3. Uniform convergence: when can we achieve PAC learning?
4. VC-dimension: how do we quantify the complexity of a problem?
5. Model selection: how do we choose the best learning model?

# 6-12: supervised learning

6. Linear binary classification
7. Linear regression
8. SVMs
9. Kernel methods and random forests
10. Boosting and PCA
11. Neural network models and gradient descent
12. Backpropagation and CNNs

# 13-15: unsupervised and reinforcement learning

13. Clustering models and algorithms: grouping objects without labels
14. Reinforcement learning: learning to act by trial and error

# A general idea: when not to learn

In this course, we will also see the limitations of learning approaches. There are some considerations we need to make before we start importing learning libraries and firing up python:

- ➔ Performance gains: is it worth it?
- ➔ Computational issues: learning (and deep learning in particular) is expensive
- ➔ Reliability: how can we ensure that the model will perform well in real life?
- ➔ Fairness: are we baking prejudices into our models?
- ➔ Explainability: can we understand the decisions the model makes?





# Biases





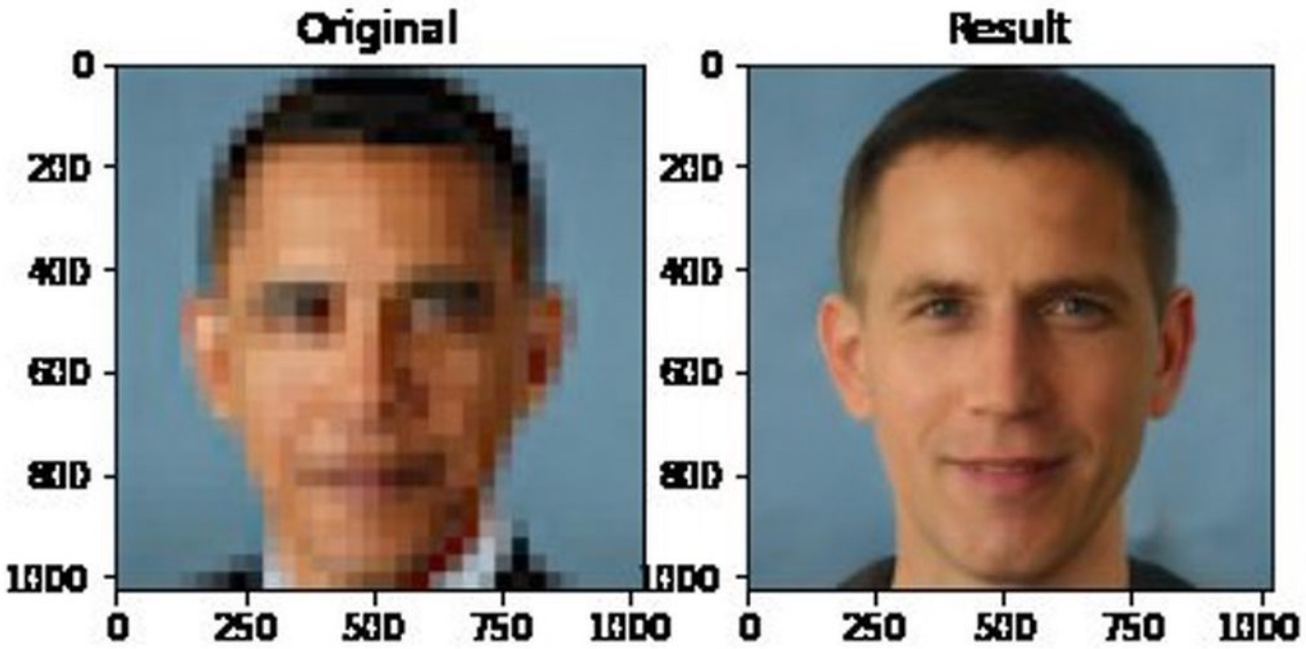







**Is this a cow?**

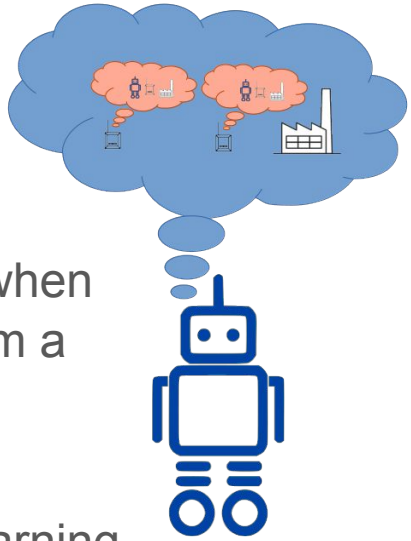
# Biases

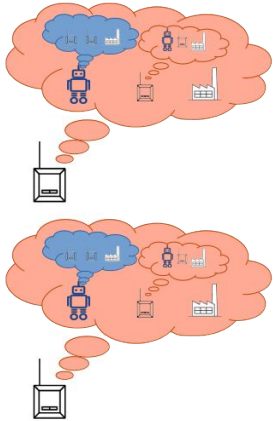


# Machine learning courses (optional, offered at DEI)

- ➔ Reinforcement Learning (1st semester)
- ➔ Adversarial Machine Learning (1st semester)
- ➔ Machine Learning for Human Data (1st semester)
- ➔ Digital Forensics and Biometrics (1st semester)
- ➔ Deep Learning (2nd semester)
- ➔ Natural Language Processing (2nd semester)

# My own research





My main line of research is on **effective communication**, i.e., a new way to decide when and what to transmit to help an agent perform a control task

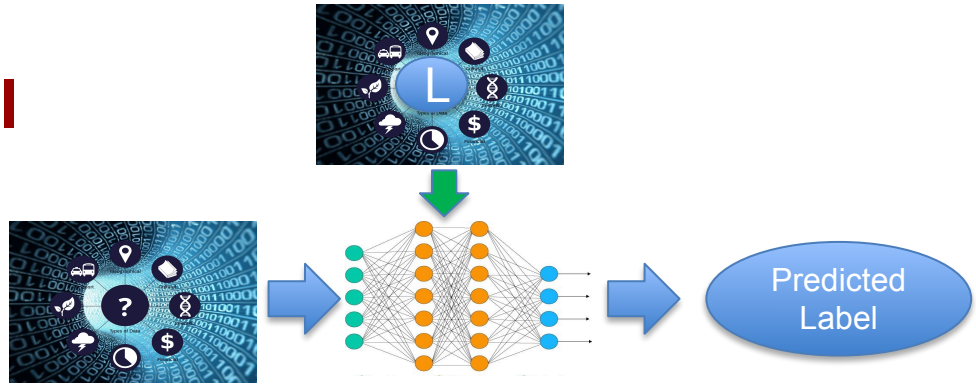
It involves a combination of unsupervised learning (to efficiently encode information) and reinforcement learning (to make decisions), combined with Markov theory, game theory, and classical optimization

# Part 2:
# The statistical learning model



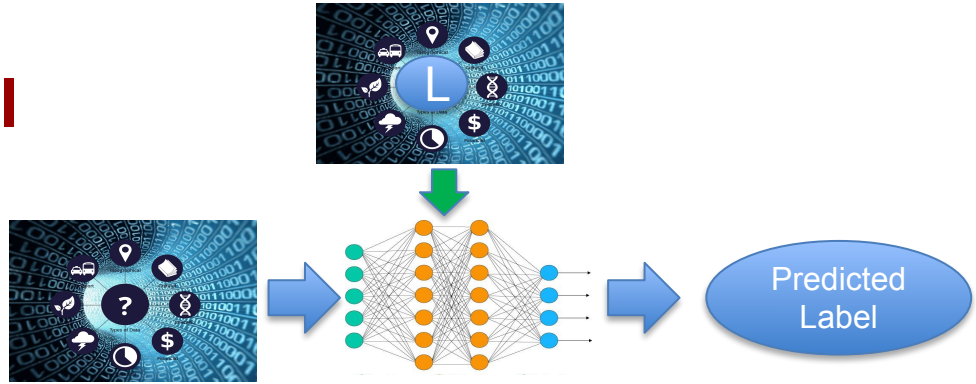

UNIVERSITÀ
DEGLI STUDI
DI PADOVA

# Supervised learning model




- Domain set (or instance space) $X$
  - $x \in X$ is a domain point or instance
  - Usually, $x$ is represented by a tensor of *features*
- Label set $Y$
  - Simplest case: binary classification, $Y = \{0, 1\}$
- Training set $S$
  - Finite sequence of labeled points $((x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m))$
  - Each point $x_i$ is associated to its label $y_i$

# Supervised learning model



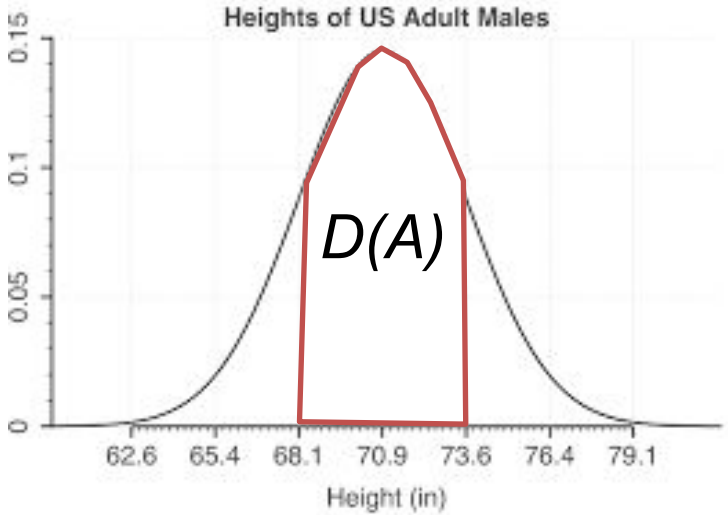

- Prediction rule $h : X \rightarrow Y$
  - The learner's output (hypothesis)
  - $A(S)$: hypothesis produced by algorithm $A$ when it is fed training set $S$
- Data generation model
  - $D$ is a distribution over $X$ **(unknown to the machine learning algorithm)**
  - Instances are labeled according to $f : X \rightarrow Y$ **(unknown to the ML algorithm)**
  - Training set: sampling according to $D, y_i = f(x_i)\ \forall x_i \in S$
- Success metric
  - Classifier error: probability of predicting the wrong label over $D$

# Events and data generating distributions

Samples $x \in X$ are produced according to a Gaussian distribution $D$

Event $A : x \in [68.1, 73.6]$

$D(A)$ is the probability of observing a point in subset $A$




$$\pi(x) = \begin{cases} 1, & \text{if } 68.1 < x < 73.6; \\ 0, & \text{otherwise.} \end{cases}$$

$$P_{x \sim D}[\pi(x) = 1] = D(A)$$

# Loss functions

- ➔ Assume a domain subset $A \subset X$
- ➔ $A$ is an event expressed by $\pi : X \to \{0, 1\}$, i.e., $A = \{x \in X : \pi(x) = 1\}$
- ➔ We get $P_{x \sim D}[\pi(x) = 1] = D(A)$

Error of prediction rule $h : X \to Y$

$$L_{D,f}(h) \stackrel{\text{def}}{=} P_{x \sim D}[h(x) \neq f(x)] = D(x : h(x) \neq f(x))$$

$L_{D,f}(h)$ (often written just as $L_D(h)$) is called true loss, true risk, or generalization loss

# Empirical Risk Minimization (ERM)

- ➔ Learner outputs $h_S : X \to Y$
- ➔ **Goal: find $\mathbf{h}^*$ that minimizes $\mathbf{L_{D,f}(h)}$**
- ➔ Both $D$ and $f$ are unknown!

ERM: we minimize the loss on the training set $L_S(h) = \frac{\sum_{i=1}^{m} |h(x_i) - y_i|}{m}$

This works if we use a 0-1 loss: the definition can be generalized

The empirical risk is also called training error or training loss

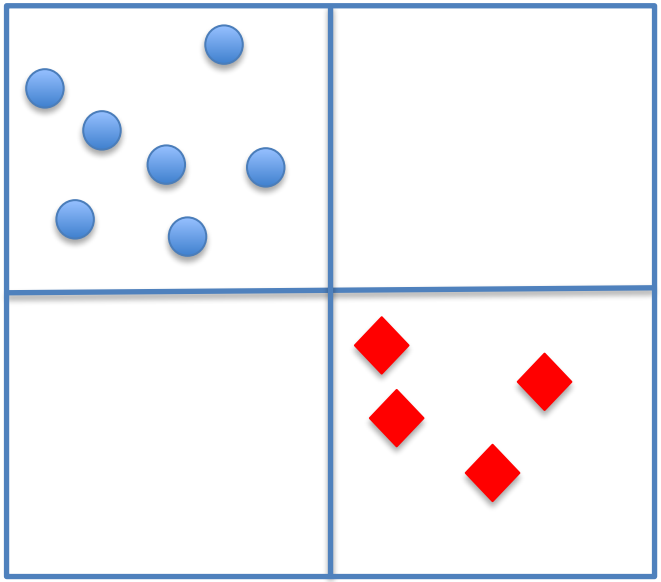
# Can we trust training error?

Let us consider the training set S in the figure, with red squares indicating class 1

$h_s(x_1,x_2)=I(x_1>0)$

This function is 1 if the point is on the right side.

$L_S(h_s)=0$

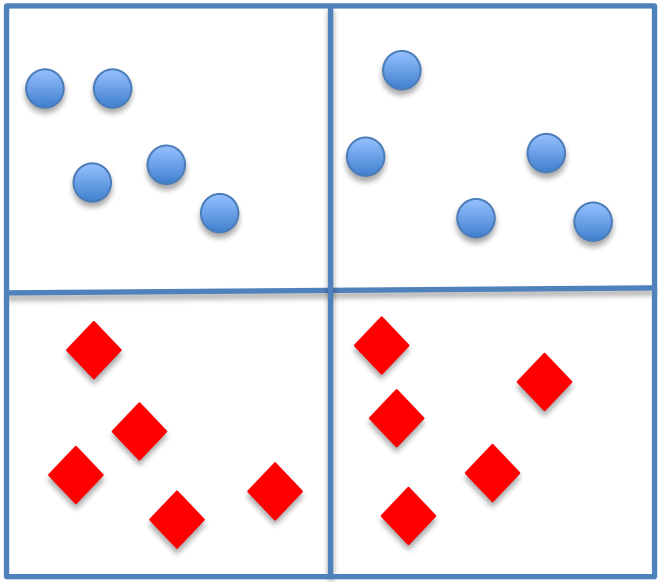


# Can we trust training error?

The real data distribution is different: X is drawn uniformly, but the rule is orthogonal

$f(x_1,x_2)=I(x_2<0)$

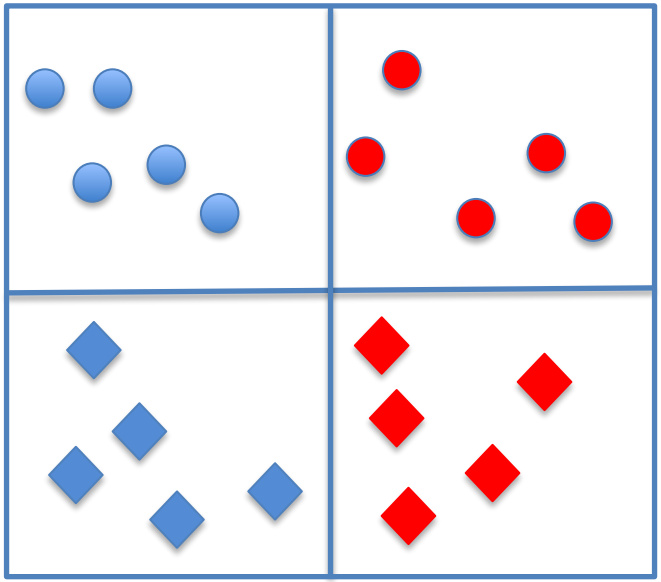
This function is 1 if the point is on the lower side.



# Can we trust training error?

In this case, our loss is exactly ½, the same as a completely random guess!

When does ERM lead to good generalization performance (i.e., a low true error?)



# Hypothesis classes

We can apply ERM over a *restricted* class $H$ of possible hypotheses

$h \in H$ is a function $h : X \to Y$

Restricted ERM finds $h^* \in \arg\min_{h \in H} L_S(h)$

There might be multiple solutions

How can we choose a class that avoids overfitting?

# Assumptions

1. Let us start with a *finite* class $H$
2. Let $h_S = \mathrm{ERM}_H \in \arg\min_{h \in H} L_S(h)$

We need two further assumptions:

3. We are lucky, and have *realizability*: $f \in H$, i.e., $\exists h^* \in H : L_{D,f}(h^*) = 0$
4. IID assumption: samples are drawn independently from $D$

Can we learn $h^*$ with ERM?